Architecture Big Data appliquée à l'IA

Séance 3 : traitement Big Data

Séance 3 : traitement Big Data

Support de cours, fil conducteur ShopFlow.
Objectif : savoir choisir entre **une grosse machine** et **un cluster**, et entre Python multiprocessus, Polars, Ray et Spark, à partir de mesures plutôt que d'intuitions.

Plan :

1. Pourquoi (et quand) distribuer
2. La mémoire pendant un traitement
3. Python sur une grosse machine
4. De MapReduce à Spark : l'exécution distribuée
5. Ray
6. Polars
7. Grille de choix

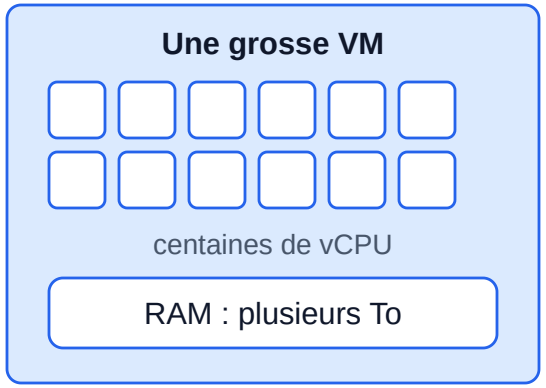
1. Pourquoi (et quand) distribuer

1.1 Monter ou s'étendre

- **Scale up** (verticalement) : une machine plus grosse. Aujourd'hui, une seule VM cloud peut avoir des centaines de vCPU et plusieurs To de RAM. Simple, pas de réseau entre les tâches, mais un plafond et un point de panne unique.
- **Scale out** (horizontalement) : plusieurs machines. Pas de plafond théorique, tolérance aux pannes possible, mais il faut **coordonner** : découper le travail, déplacer des données sur le réseau, gérer les pannes.

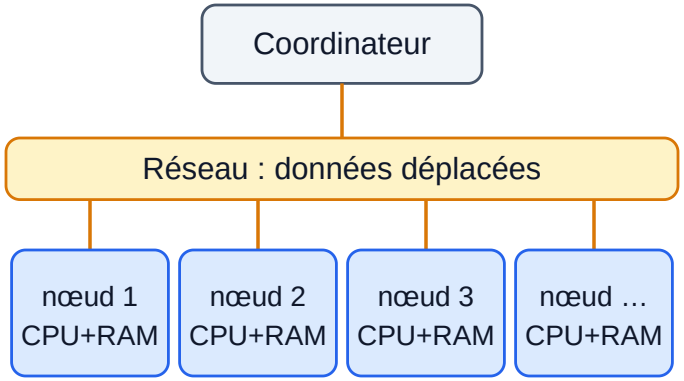
1.1 Monter ou s'étendre (suite)

Scale up (verticalement)



+ simple
+ pas de réseau entre tâches
− un plafond matériel
− un point de panne unique

Scale out (horizontalement)



+ pas de plafond théorique
+ tolérance aux pannes possible
− coordonner : découper le travail, déplacer les données, gérer les pannes

Schéma : à gauche tout tient dans une machine ; à droite le réseau et le coordinateur apparaissent, avec leurs coûts.

1.2 Les limites : ajouter des machines ne divise pas toujours le temps

Intuition : 10 cuisiniers épluchent 10 fois plus vite les légumes, mais le gâteau ne cuit pas plus vite dans le four. Une partie du travail ne se découpe pas.

Exemple ShopFlow. Le calcul nocturne des indicateurs de fraude dure **100 min** sur une machine :

- **10 min** ne se découpent pas (lire la liste des fichiers, fusionner le résultat final, écrire un seul fichier) ;
- **90 min** se découpent (chaque machine traite une partie des paiements).

Machines	Partie non découpable	Partie découpée	Durée totale	Gain
1	10 min	90 min	100 min	×1
10	10 min	9 min	19 min	×5
100	10 min	0,9 min	≈ 11 min	×9
1 000	10 min	0,09 min	≈ 10 min	×10

1.2 Les limites : ajouter des machines ne divise pas toujours le temps (suite)

On ne descend **jamais sous 10 min** : le gain plafonne à ×10, quel que soit le nombre de machines. C'est la **loi d'Amdahl** : si une part `s` du travail ne se découpe pas, le gain maximal est `1 / s` (ici `s` = 10 %, donc ×10 ; avec 5 %, ×20).

Et en pratique, c'est pire : chaque machine ajoutée coûte aussi de la **coordination**.

- **Se répartir le travail** : un chef d'orchestre doit envoyer les tâches et attendre la plus lente.
- **Échanger des données** : pour compter les paiements par carte, tous les paiements d'une même carte doivent arriver sur la même machine. Ils voyagent donc par le réseau, bien plus lent que la RAM (séance 1). Cet échange s'appelle le **shuffle**.
- **Emballer les données** pour le réseau (sérialisation, partie 2.2).

Au-delà d'un certain nombre de machines, ces coûts dépassent le gain : le traitement peut même devenir **plus lent**.

1.2 Les limites : ajouter des machines ne divise pas toujours le temps (suite)

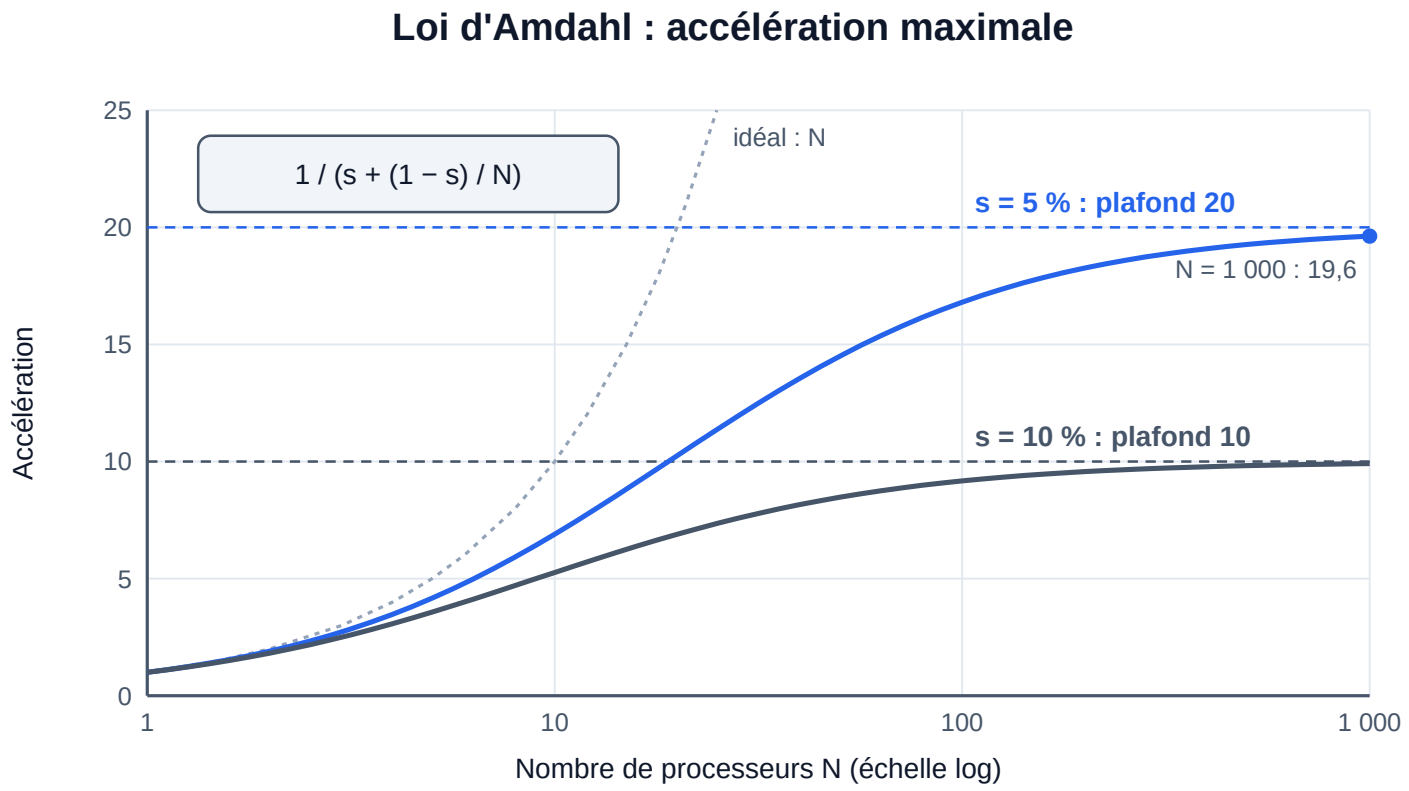


Schéma : plus on ajoute de processeurs, moins chaque processeur supplémentaire fait gagner ; la courbe s'aplatit sous un plafond (×20 si 5 % du travail ne se découpe pas, ×10 si 10 %).

1.3 Un seul ordinateur bien programmé bat souvent un cluster

En 2015, trois chercheurs (McSherry, Isard et Murray) ont posé une question simple : **à partir de combien de cœurs un système distribué bat-il un seul cœur bien programmé ?** Ils appellent ce seuil le **COST** (*Configuration that Outperforms a Single Thread*).

Leur exemple : calculer PageRank (le classement des pages façon Google) sur un graphe Twitter de 1,5 milliard de liens, 20 itérations.

Configuration	Durée
Spark GraphX, **128 cœurs**	environ 420 s
Un seul thread sur un ordinateur portable, code simple	environ 300 s
Un seul thread, données mieux rangées en mémoire	environ 110 s

Le cluster de 128 cœurs est **plus lent** qu'un portable. Pourquoi ? Le cluster passe une grande partie de son temps à coordonner et à échanger des données (partie 1.2), alors que le portable ne fait que calculer.

1.3 Un seul ordinateur bien programmé bat souvent un cluster (suite)

Ce que ça ne veut pas dire : les clusters restent indispensables quand les données **ne tiennent pas** sur une machine ou qu'il faut **résister aux pannes**.

Ce que ça veut dire, et la règle du TP : avant de distribuer, mesurer d'abord la solution simple sur une machine (pandas, Polars, `multiprocessing`). On ne passe au cluster que si on a une bonne raison chiffrée.

1.3 Un seul ordinateur bien programmé bat souvent un cluster (suite)

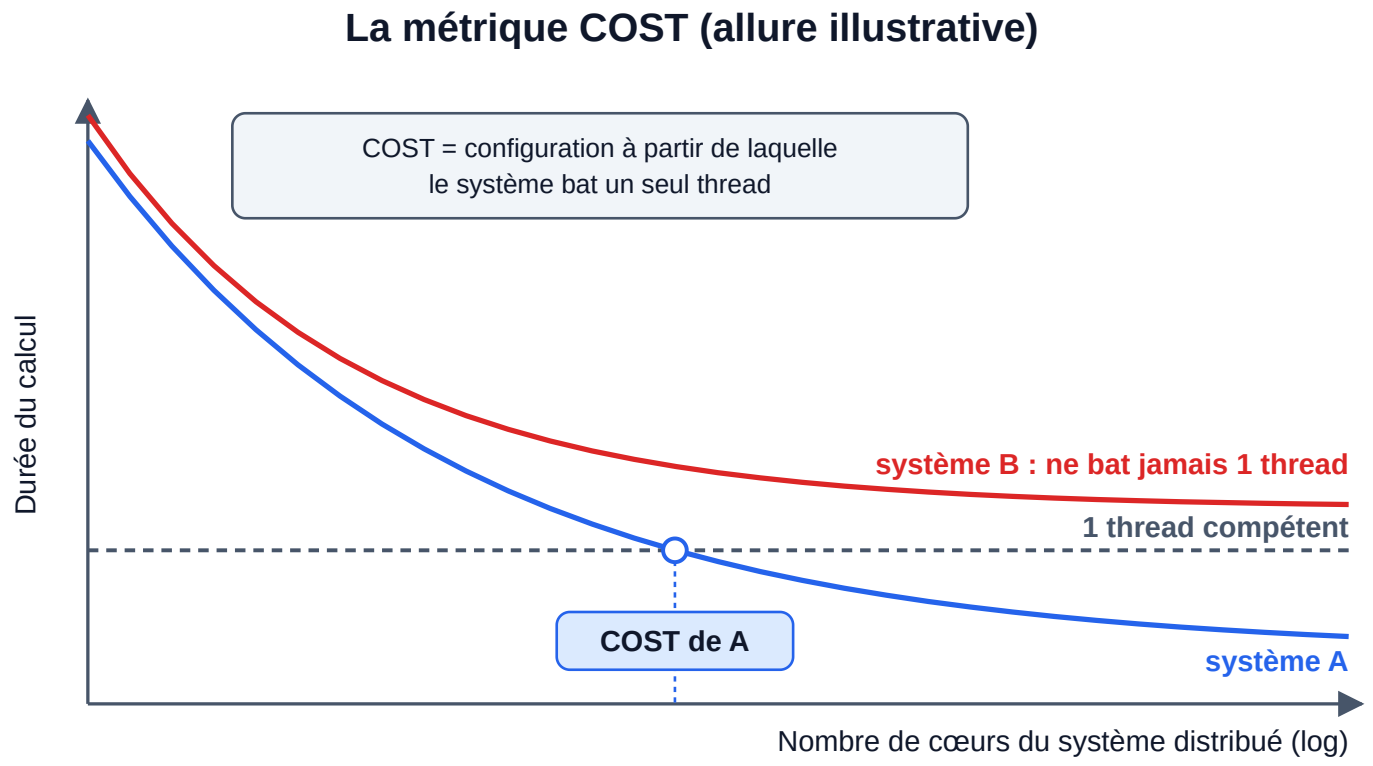


Schéma (allure illustrative) : la ligne horizontale est le temps d'un seul thread. Le système A finit par passer en dessous, mais seulement avec beaucoup de cœurs ; le système B ne la passe jamais.

Sources : 1.3 Un seul ordinateur bien programmé bat souvent un cluster

Sources : Kleppmann, *Designing Data-Intensive Applications*, O'Reilly, 2017, chap. 1 (« Scalability ») et chap. 10. ; Amdahl, *Validity of the single processor approach to achieving large scale computing capabilities*, AFIPS 1967 ; présentation pédagogique dans Hennessy & Patterson, *Computer Architecture*, 6e éd., 2017, chap. 1. ; McSherry, Isard, Murray, *Scalability! But at what COST?*, HotOS 2015 (tableau PageRank sur le graphe twitter_rv).

2. La mémoire pendant un traitement

On reprend la hiérarchie mémoire de la séance 1, vue cette fois depuis un programme qui s'exécute.

2.1 Où vivent les données

Endroit	Ce qu'on y met	Ce qui arrive quand c'est plein
Caches CPU	Les valeurs en cours de calcul	Le CPU attend la RAM (*cache miss*)
RAM du processus	DataFrames, tables de hachage, tris	**Spill** sur disque, ou arrêt du processus (*Out Of Memory*)
Page cache de l'OS	Les fichiers récemment lus	L'OS évince les pages les plus anciennes
Disque local / SSD	Fichiers temporaires, spill, shuffle	Erreur « no space left on device »
Stockage objet (GCS)	Données sources et résultats	Pratiquement illimité

- Le **page cache** explique qu'une deuxième exécution soit souvent plus rapide : les fichiers sont déjà en RAM. Pour mesurer honnêtement, on compare des exécutions « à froid » et « à chaud ».
- Un **cache applicatif** (`df.cache()` dans Spark) garde un résultat intermédiaire en mémoire pour le réutiliser. Utile s'il est relu plusieurs fois, nuisible sinon (il prend de la RAM au reste).

2.2 Le coût caché de la sérialisation

- Pour passer des données d'un processus à un autre (ou d'une machine à une autre), il faut les **sérialiser** (transformer en octets) puis les désérialiser. En Python, `pickle` sur des objets Python est lent et volumineux.
- **Apache Arrow** définit un format colonnaire **en mémoire** standard : des tableaux contigus par colonne, identiques quel que soit le langage. Deux outils qui parlent Arrow (Polars, pandas 2+, Spark, Ray, DuckDB) peuvent s'échanger des données **sans copie ni conversion**.

2.2 Le coût caché de la sérialisation (suite)

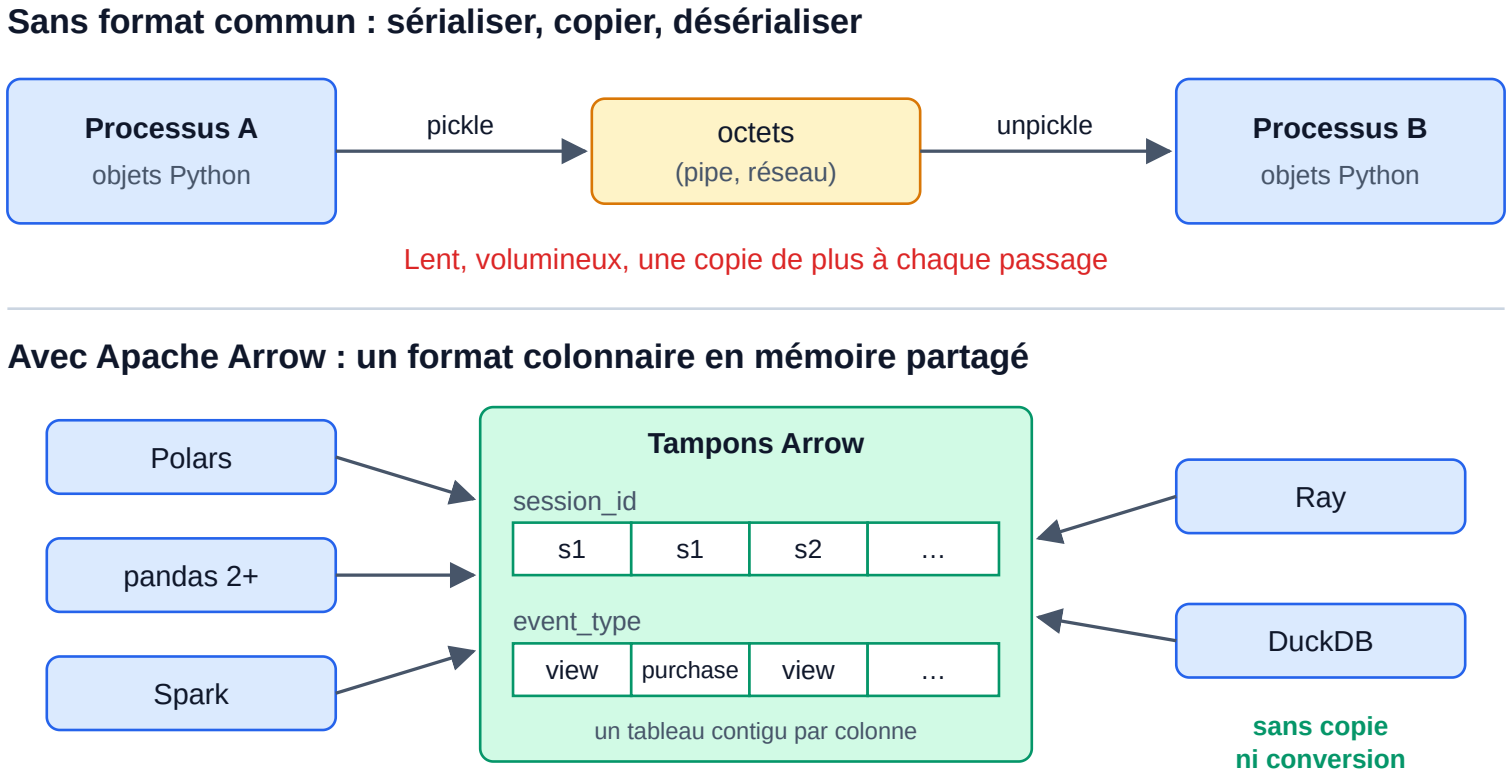


Schéma : en haut, chaque passage sérialise et copie ; en bas, les outils lisent les mêmes tableaux contigus par colonne.

2.3 Observer

Il est important pour chaque ressource (CPU, mémoire, disque, réseau), de regarder **Utilisation**, **Saturation** et **Erreurs**. Outils du TP : `htop`, `free -h`, `/usr/bin/time -v` (pic de mémoire résidente), `psutil`, l'interface web de Spark.

Sources : 2.3 Observer

Sources : Apache Arrow : columnar format. ; Spark, Tuning : memory management overview. ; Gregg, *Systems Performance*, 2e éd., Addison-Wesley, 2020, chap. 2 et 7 ; la méthode USE.

3. Python sur une grosse machine

3.1 Threads, processus et GIL

- Un **thread** partage la mémoire de son processus ; un **processus** a sa propre mémoire.
- En CPython classique, le **GIL** (*Global Interpreter Lock*) empêche deux threads d'exécuter du bytecode Python en même temps : les threads accélèrent les entrées-sorties (réseau, disque), pas le calcul pur Python.
- Les bibliothèques écrites en C, C++ ou Rust (NumPy, Arrow, Polars) relâchent le GIL pendant leurs calculs : elles exploitent plusieurs cœurs même depuis un seul thread Python.
- CPython 3.13 a introduit une version **sans GIL** expérimentale (PEP 703) ; depuis Python 3.14, elle est officiellement supportée (PEP 779) mais reste une option, pas la version par défaut.

3.2 `multiprocessing` et `concurrent.futures`

- `multiprocessing` permet de lancer **plusieurs processus Python en même temps** sur la machine. Chacun a sa propre mémoire et son propre GIL : ils calculent vraiment en parallèle, sur plusieurs cœurs.
- `concurrent.futures` permet de **soumettre des tâches** à un groupe de processus (un *pool*), puis de récupérer les résultats. Avec `ProcessPoolExecutor`, ce groupe est créé par `multiprocessing` en coulisse : on ne gère pas les processus soi-même.

C'est la combinaison utilisée au TP 3 :

```
from concurrent.futures import ProcessPoolExecutor

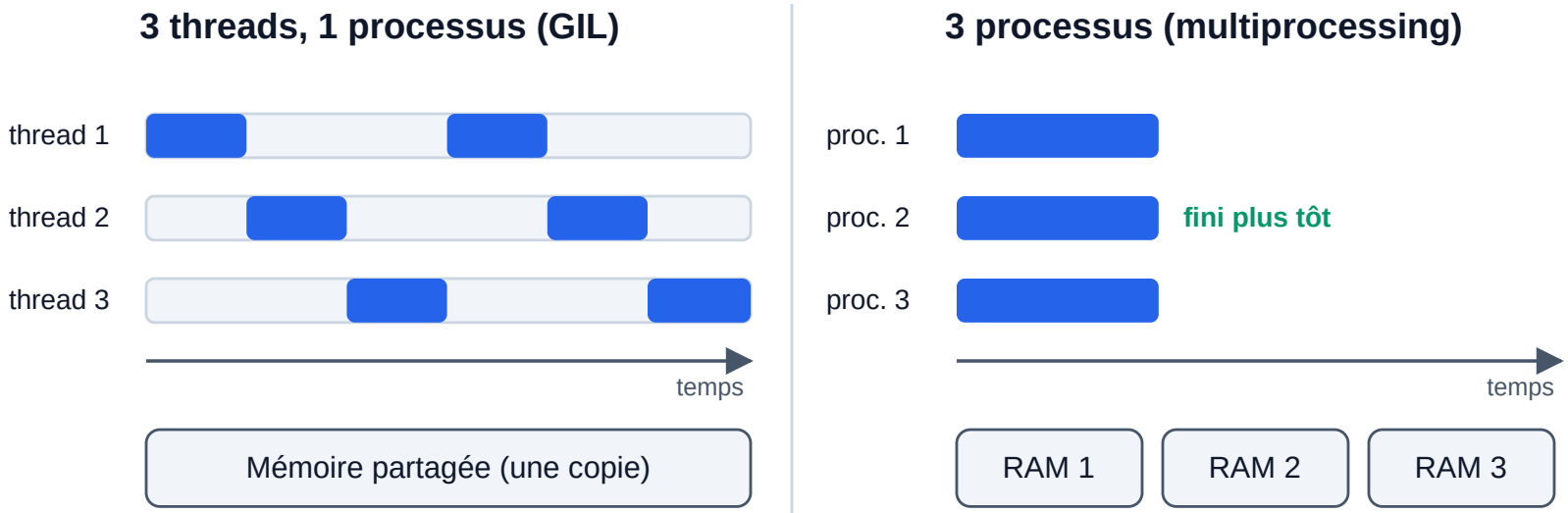
if __name__ == "__main__":
    with ProcessPoolExecutor(max_workers=8) as pool:      # 8 processus lancés
        resultats = list(pool.map(traiter_un_fichier, fichiers))   # 1 tâche par fichier
```

3.2 `multiprocessing` et `concurrent.futures` (suite)

À retenir

- Chaque processus a sa propre mémoire : **la RAM utilisée est multipliée** par le nombre de processus.
- Idéal quand le travail se découpe en **morceaux indépendants**, par exemple un fichier par mois. Renvoyer un petit résultat (pas un gros DataFrame), car tout ce qui passe entre processus est copié.

3.2 `multiprocessing` et `concurrent.futures` (suite)



- Un seul thread exécute du bytecode Python à la fois
- Utile pour les E/S (réseau, disque), pas pour le calcul pur Python

- Calcul en parallèle sur 3 cœurs
- RAM totale multipliée par 3
- Arguments et résultats sérialisés avec pickle

Schéma : pour un calcul pur Python, les threads se relaient (un seul exécute à la fois) alors que les processus avancent ensemble, au prix d'une RAM par processus.

Sources : 3.2 `multiprocessing` et `concurrent.futures`

Sources : Python, `multiprocessing` et `concurrent.futures` ; PEP 703 ; PEP 779.

4. De MapReduce à Spark

4.1 MapReduce (Google, 2004)

- Le programmeur écrit deux fonctions : **map** (transformer chaque enregistrement en paires clé-valeur) et **reduce** (combiner les valeurs d'une même clé). Le système gère le découpage, la distribution, le **shuffle** entre les deux et les pannes (en relançant les tâches).
- Limite : chaque étape écrit ses résultats sur disque. Un algorithme itératif (ML, graphes) relit tout à chaque itération.

4.1 MapReduce (Google, 2004) (suite)

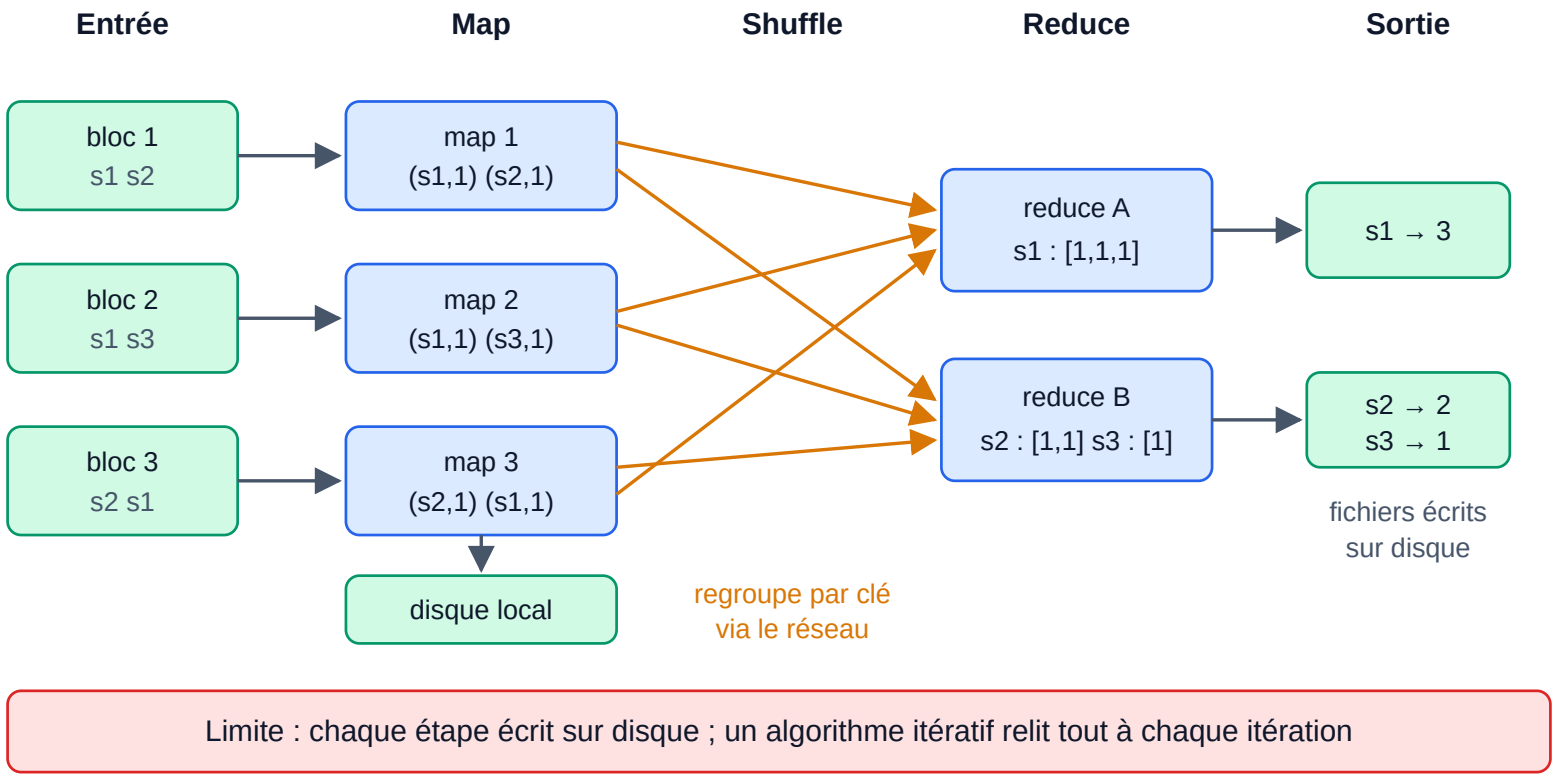


Schéma (d'après Dean et Ghemawat, 2004) : compter les événements par session ; le shuffle regroupe toutes les paires d'une même clé sur un même reduce, et chaque étape passe par le disque.

4.2 Spark : garder en mémoire, se souvenir du calcul

- Un **RDD** (*Resilient Distributed Dataset*) est une collection partitionnée et immuable qui connaît son **lignage** : la suite de transformations qui l'a produite. En cas de panne, on recalcule la partition perdue au lieu de répliquer les données.
- Les DataFrames (séance 2) s'appuient sur ce modèle et ajoutent l'optimiseur Catalyst.

4.3 Comment un job s'exécute

Concept	Définition
Driver	Le processus qui construit le plan et pilote l'exécution
Executor	Un processus sur un nœud du cluster qui exécute des tâches et garde des données en mémoire
Partition	Un morceau des données ; une **tâche** traite une partition
Stage	Un groupe de tâches sans échange de données ; une frontière de stage = un **shuffle**
Shuffle	Redistribution des données par clé entre executors (`groupBy` , `join`) : écriture sur disque local + réseau

4.3 Comment un job s'exécute (suite)

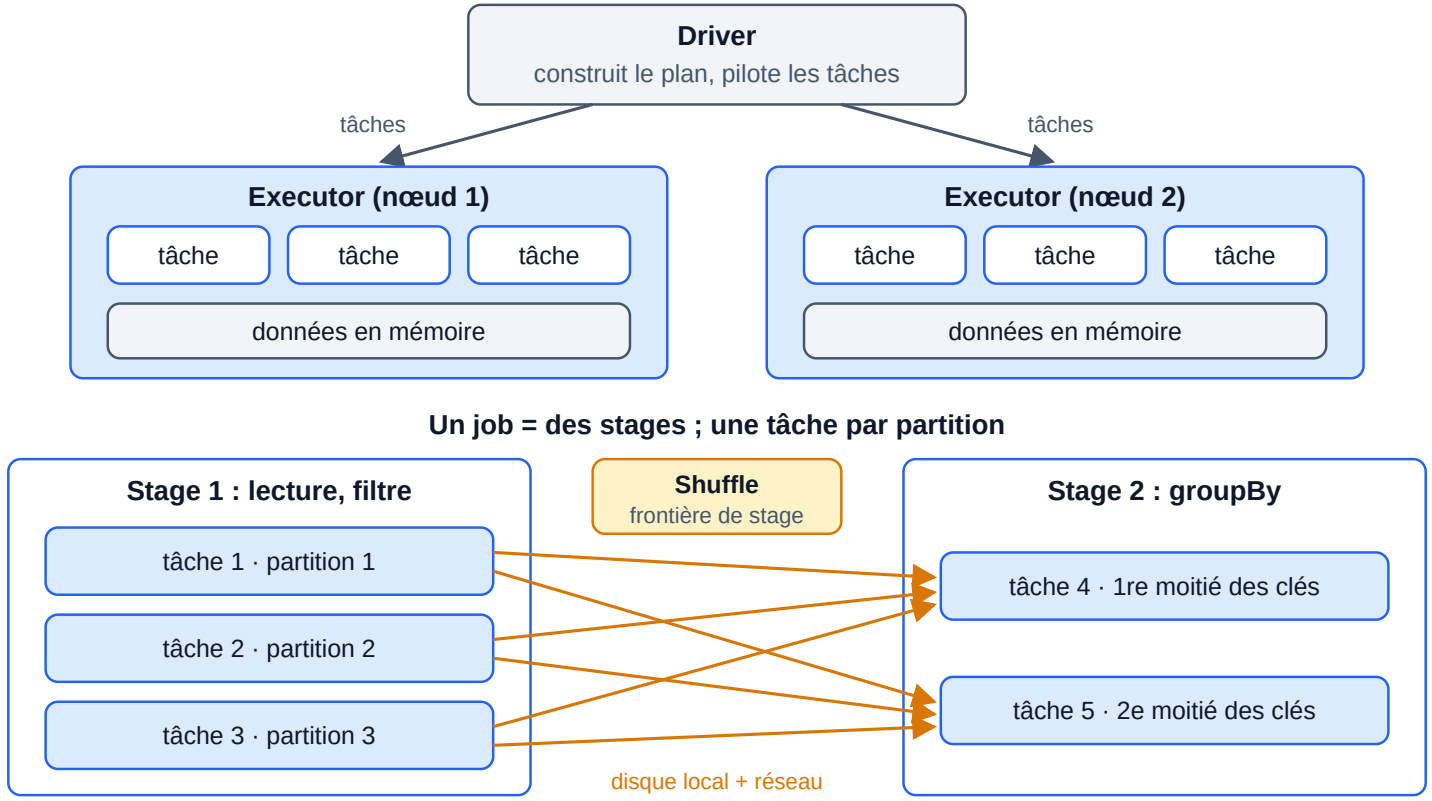
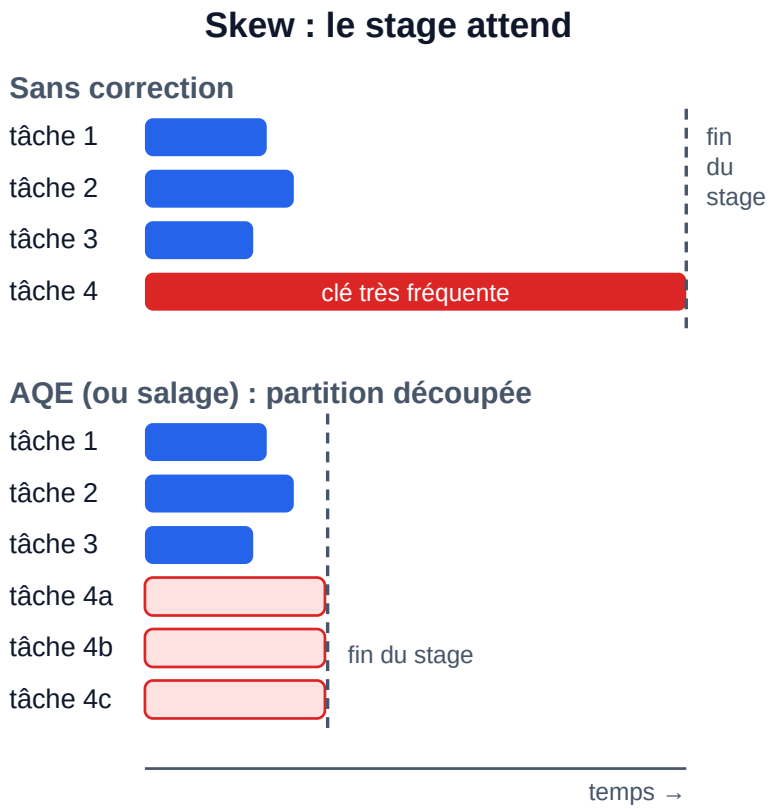


Schéma (d'après la page Cluster mode overview de Spark) : en haut qui exécute quoi ; en bas, une tâche par partition et un shuffle qui marque la frontière entre deux stages.

4.4 Les problèmes classiques

- **Skew** (déséquilibre) : une clé beaucoup plus fréquente que les autres (une carte frauduleuse très active, le client « invité ») fait qu'une tâche traite bien plus de données que les autres ; tout le stage attend la dernière.
  Remèdes : **AQE** (*Adaptive Query Execution*, activée par défaut depuis Spark 3.2) découpe automatiquement les partitions déséquilibrées dans les jointures ; sinon, le **salage** (ajouter un suffixe aléatoire à la clé).
- **Spill** : une tâche n'a pas assez de mémoire pour son tri ou sa table de hachage et écrit sur disque. Visible dans l'interface Spark (*Spill (Memory)*, *Spill (Disk)*).
- **Broadcast join** : si une table est petite (une dimension), on l'envoie à chaque executor plutôt que de redistribuer la grande.

4.4 Les problèmes classiques (suite)



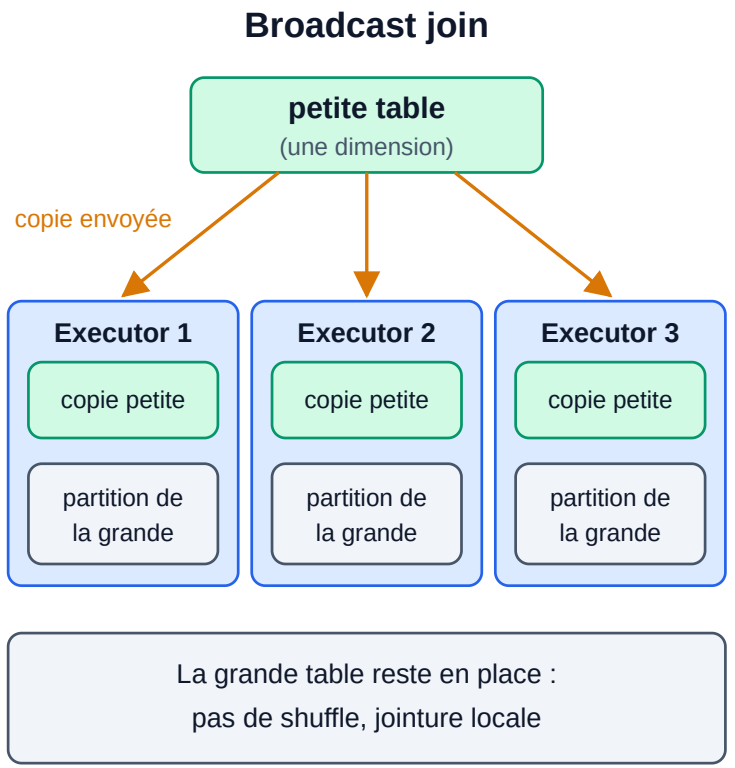


Schéma : à gauche, la tâche de la clé très fréquente fixe la fin du stage, et la découper la ramène vers les autres ; à droite, la petite table est copiée sur chaque executor et la grande ne bouge pas.

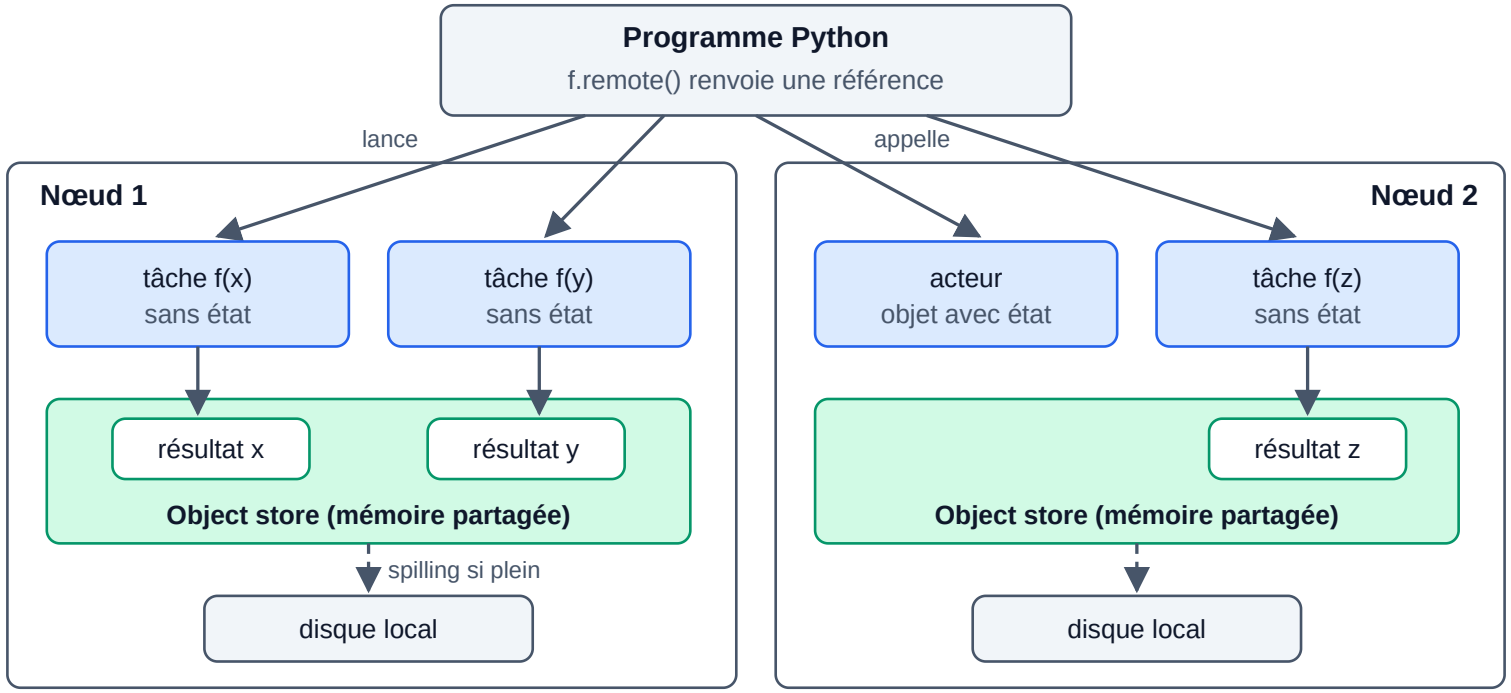
Sources : 4.4 Les problèmes classiques

Sources : Dean & Ghemawat, *MapReduce: Simplified Data Processing on Large Clusters*, OSDI 2004. ; Zaharia et al., *Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*, NSDI 2012. ; Spark, Performance tuning (AQE, broadcast) ; Cluster mode overview ; Monitoring and instrumentation. ; Chambers & Zaharia, *Spark: The Definitive Guide*, O'Reilly, 2018, chap. 15 à 19.

5. Ray

- Ray est né à Berkeley (RISELab) pour les applications d'IA, qui mélangent calcul de données, entraînement et service de modèles.
- Deux primitives : les **tâches** (une fonction Python décorée `@ray.remote`, exécutée n'importe où dans le cluster) et les **acteurs** (un objet Python avec état, qui vit sur un nœud).
- Un **object store** en mémoire partagée par nœud stocke les résultats ; les objets sont transmis par référence. Quand il est plein, Ray **déverse sur disque** (*object spilling*).
- **Ray Data** : une bibliothèque de traitement de données en flux (*streaming execution*) sur des blocs Arrow, utilisée surtout pour la préparation de données et l'**inférence batch** sur CPU et GPU.
- Positionnement : plus souple que Spark pour du Python arbitraire et du ML ; moins mature que Spark pour le SQL et les grosses jointures.

5. Ray (suite)



Les résultats restent dans l'object store ; le programme manipule des références
et récupère la valeur avec ray.get(référence)

Schéma : les tâches écrivent leurs résultats dans l'object store de leur nœud, l'acteur garde son état sur le sien ; le programme ne manipule que des références.

5. Ray (suite)

```
import ray
ray.init()
ds = ray.data.read_parquet("gs://.../clickstream/")
stats = ds.groupby("session_id").count()
```

Sources : 5. Ray

Sources : Moritz et al., *Ray: A Distributed Framework for Emerging AI Applications*, OSDI 2018 ; documentation Ray Data ; Ray Dashboard.

6. Polars

- Bibliothèque DataFrame écrite en **Rust**, fondée sur le format Arrow, **multithread par défaut** (tous les cœurs de la machine).
- **API lazy** : `pl.scan_parquet(...)` construit un plan ; l'optimiseur pousse les filtres et ne lit que les colonnes utiles ; `.collect()` exécute. On peut afficher le plan avec `.explain()`.
- **Moteur streaming** : traite les données par morceaux, ce qui permet de travailler sur des données plus grandes que la RAM (`collect(engine="streaming")`).
- Limite : une seule machine. Pour des volumes qui tiennent sur un disque local, c'est souvent la solution la plus rapide et la plus simple (voir le COST, bloc 1).

6. Polars (suite)

```
import polars as pl
res = (pl.scan_parquet("clickstream/*.parquet")
         .group_by("session_id")
         .agg(pl.len().alias("n_events"))
         .collect(engine="streaming"))
```

6. Polars (suite)

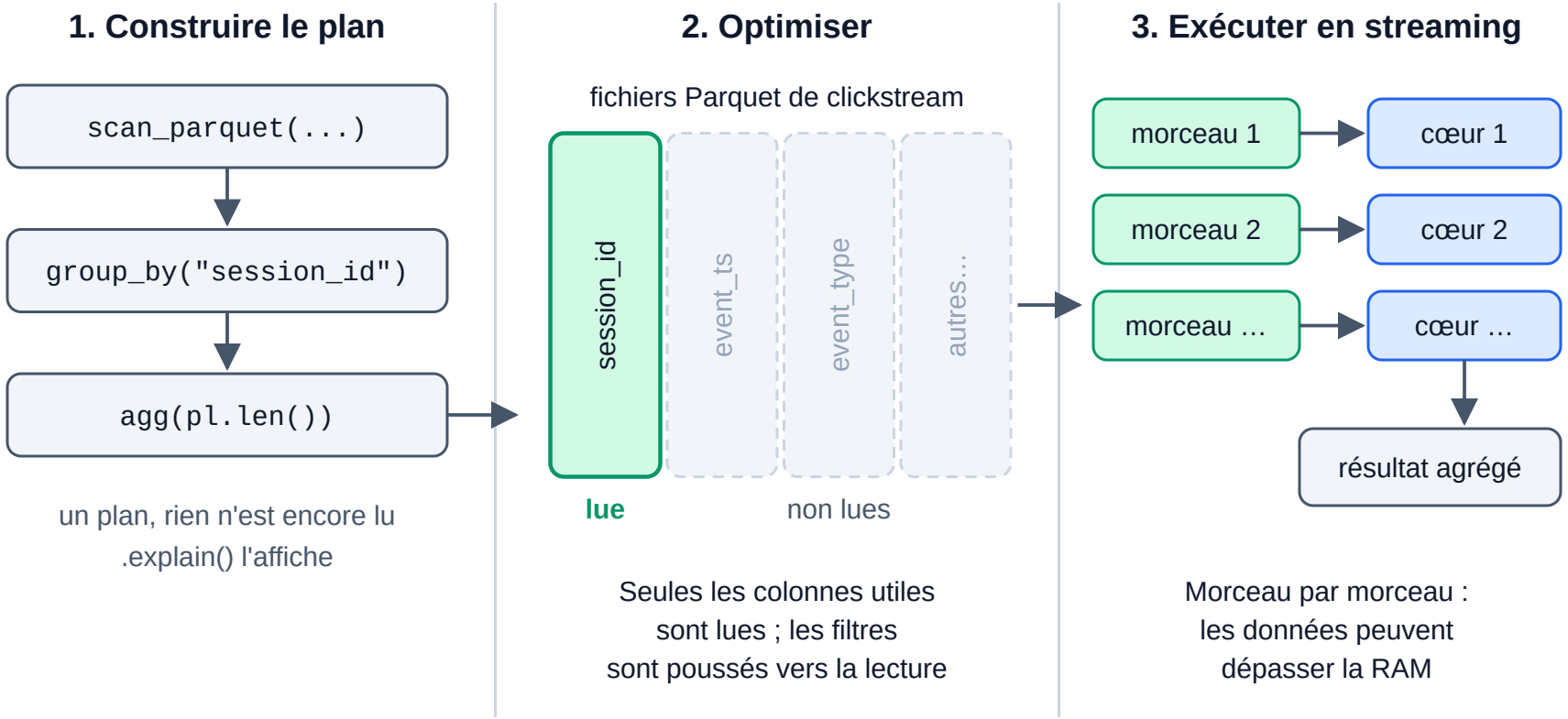


Schéma : pour l'exemple ci-dessus, rien n'est lu avant collect(), seule la colonne session_id est lue, puis les morceaux sont traités sur tous les cœurs.

Sources : 6. Polars

Sources : Polars user guide ; lazy API ; streaming.

7. Grille de choix

Situation	Choix raisonnable
Données < RAM d'une machine, calcul relationnel	Polars (ou DuckDB) sur une machine
Données découpables en fichiers indépendants, code Python existant	`multiprocessing` sur une grosse machine
Données > disque/RAM d'une machine, jointures et agrégations lourdes, SQL	Spark sur un cluster
Python arbitraire, ML, inférence batch sur GPU	Ray
Besoin de tolérance aux pannes sur des heures de calcul	Spark ou Ray (relance des tâches perdues)

Cette grille sera **confrontée aux mesures du TP** : chaque binôme la complète avec ses chiffres.